

HelixPlay

HelixPlay

あらゆるGPU搭載マシンをクラウドゲーミング端末に変える

概要

HelixPlayは、自己ホスティング可能なクラウドゲーミングプラットフォームです。GPUを搭載した任意のマシンをリモートストリーミングホストに変え、デスクトップ、モバイル、テレビ、ブラウザクライアントに据置型ゲーム機並みのゲーム体験を提供します。46のサブモジュールから成るGo中心のモノレポとして構築され、パートナー向けのホワイトラベル化にも対応しています。

短い説明

HelixPlayは、自己ホスティング可能でオープンなホワイトラベル対応のクラウドゲーミングプラットフォームです。GPU搭載マシンをリモートストリーミングホストに変え、WebRTC/QUICを通じてデスクトップ、モバイル、テレビ、ブラウザクライアントに据置型ゲーム機並みのゲーム体験を提供します。Goをコアとし、Wails/Flutter/Angularのクライアントスタックを採用しています。

長い説明

HelixPlayは、46のGitサブモジュールから構成されるGo中心のモノレポとして開発されたクラウドゲーミングプラットフォームです。既に所有しているゲーミングPCをストリーミングホストに変え、デスクトップ、モバイル、テレビ、ブラウザクライアントに据置型ゲーム機並みの体験を提供します。自己ホスティング可能でオープン、パートナー向けのホワイトラベル化にも対応しています。そのコンセプトは明確です。あなたのハードウェア、あなたのサービス、あなたのブランド。第三者のクラウドは介在しません。

その最大の設計上の特徴は、トリプルスタックのクライアント統合です。これは、他のあらゆる部分で効果を発揮する大胆なアーキテクチャ上の賭けです。Wailsデスクトップアプリ、Flutterモバイル/テレビアプリ、Angularウェブクライアントはすべて、単一のGoコア上に構築されており、ブラウザ向けにWASMにコンパイルされています。そのため、動作は一度記述するだけで、すべてのプラットフォームで共有され、三つに分岐することはありません。その下にはリアルタイムメディアパスが存在します。キャプチャ→エンコード→パケット化→送信→デコード→レンダリングという流れで、プラットフォームネイティブのキャプチャ（DXGI / ScreenCaptureKit / PipeWire）とハードウェアエンコーダ（NVENC / QSV / AMF / VideoToolbox）に接続されており、GPUが重い処理を担います。そして、WebRTC（Pion v4）、QUIC（quic-go）、低遅延を優先したカスタムUDPデータグラムを通じて転送されます。バックエンドコアはセッション、テナント、カタログ、認証を管理し、ホストエージェントはエッジでのキャプチャ、エンコード、転送を担当します。mDNS/ランデブー機能により、クライアントは手動設定なしでホストを発見できます。

HelixPlayは、ホワイトラベルSaaSを前提にゼロから設計されています。テナントごとのテーマ設定、カタログフィルタリング、OAuth2、課金機能を備えているため、パートナーは薄いリスクではなく、完全にブランディングされたサービスを立ち上げることができます。また、あらゆるサービス、データベース、ビルド、テスト、スキャンがコンテナ内で実行されるコンテナネイティブな設計となっており、プラットフォーム全体のデプロイと検証の再現性を確保しています。Helixファミリー同様、グリーンテストはモックの通過ではなく、実際のエンドユーザーが利用可能な動作を保証することを目的とした「ブラフ禁止」の原則に基づいています。

なぜ開発したのか

商用のクラウドゲーミングはクローズドで中央集権的、そしてレンタル型です。HelixPlayは、GPUマシンを持つ誰もが、第三者のサービスに依存することなく、オープンで自己ホスティング可能、ホワイトラベル対応のストリーミングホストを運用できるようにするために開発されました。

コンテンツ

なぜこれはゲームチェンジャーなのか

商用サービスが切り離してきた3つの要素を融合させた：自分で管理するハードウェア上でのセルフホスティング、3つのクライアントスタックを同時に動かす単一のGoコア、そしてホワイトラベルのマルチテナンシー。その結果、パートナーは他社のクラウドの容量を再販売してその制約の中で運用するのではなく、自社のGPU上で完全にブランディングされたクラウドゲーミングサービスを立ち上げることができる。これにより、エクスペリエンス、ユーザー、そして経済性を自ら所有することが可能になる。

革新的なポイント

- トリプルスタックのクライアント統合 — Wails、Flutter、Angularがすべて単一のGoコア（ブラウザではWASM）上で動作。デスクトップ、モバイル、TV、ウェブが3つの異なる実装ではなく、1つの実装を共有する。
- セルフホスティング可能なホワイトラベル**SaaS** — テナントごとのテーマ設定、カタログフィルタリング、OAuth2、課金機能を内蔵。プラットフォームはデモではなく、ブランド化可能な製品として提供される。
- 最新の低遅延トランスポート — WebRTC（Pion）、QUIC、カスタムUDPに加え、プラットフォームごとのハードウェアエンコーダー選択（NVENC / QSV / AMF / VideoToolbox）を採用。利便性よりも応答性を重視して最適化。
- 46のサブモジュールによる分離アーキテクチャ — コンポーネントが明確に分離され、すべてがコンテナネイティブ。各サービス、データベース、ビルド、テスト、スキャンがコンテナ内で実行される。

最大の技術的課題とその解決方法

- 異種ハードウェア間での低遅延ストリーミング。OSやGPUごとにキャプチャとエンコードの方法が異なり、遅延に対する許容度は低い。解決策として、プラットフォームに応じたキャプチャ／エンコードパス（DXGI / ScreenCaptureKit / PipeWire → NVENC / QSV / AMF / VideoToolbox）を採用し、WebRTC / QUIC / UDPを介して各マシンが最速のネイティブルートでピクセルデータを処理。
- デスクトップ、モバイル、**TV**、ウェブを横断する単一製品。トリプルスタッククライアント（Wails、Flutter、Angular）が単一のGoコアを共有し、ブラウザ向けにWASMにコンパイル。これにより、一度書かれた修正や機能が4つのプラットフォームすべてに反映され、4回の移植作業が不要に。
- マルチテナントのホワイトラベル運用。テナントごとのテーマ設定、カタログフィルタリング、OAuth2、課金機能をコアバックエンドに直接組み込むことで解決。テナントの分離とブランディングは、個別カスタマイズではなくプラットフォームの基本機能として実現。

テックスタック

- **Go（1.26.2ルート / 1.25+サブモジュール）** — 共有コアバックエンドおよびホストエージェント。ネイティブバイナリとWASMの両方にコンパイル可能な単一言語で、シングルコア・マルチクライアント設計を実現。
- **Wails v2** — デスクトップクライアント。Goコアを埋め込みWebViewにバインドし、デスクトップアプリがコアロジックを再実装することなく直接利用。

- **Flutter 3.29+** — モバイル／TVクライアント。FFIを介してGoコアを呼び出し、2つ目のバックエンドなしでスマートフォンやテレビ向けのネイティブUIを実現。
- **Angular 17+** — ウェブクライアント。同じGoコアをWASMにコンパイルして実行。ブラウザは機能を削減された環境ではなく、第一級のプラットフォームとして扱われる。
- **WebRTC / Pion v4、QUIC / quic-go、カスタムUDP** — 3種類のリアルタイムトランスポート。ネットワークやクライアントごとに最低遅延のパスを選択可能。
- **ハードウェアエンコーダー (NVENC / QSV / AMF / VideoToolbox)** および **プラットフォームキャプチャ (DXGI / ScreenCaptureKit / PipeWire)** — GPUによる高速化キャプチャ・エンコードパス。プラットフォームごとに最適なエンコード方式を選択し、CPUボトルネックを回避。
- **コンテナ (Docker / Podman)** — すべてのサービス、データベース、ビルド、テスト、スキャンがコンテナ化。システム全体のデプロイと検証を再現可能に。
- **mDNS / レンデブー** — ゼロコンフィグのホスト検出。クライアントがネットワーク上のストリーミングホストを自動的に発見。

コンテンツ

ステータスと正確性に関する注意事項

- **ステータス:** ベータ版。READMEに記載されているレイテンシ目標 (LAN: 99.9パーセンタイルで30ミリ秒以下／WAN: 同50ミリ秒以下)、「コンソールクラス／PS4 Proクラス」という表現、およびテストマトリックスのセル数は、いずれもプロジェクトが独自に設定した設計目標であり、第三者によるベンチマークは行われておらず、その旨を明記した上で提示しています。
- **ライセンス:** 未定。GitHubおよびAPIによる確認ではLICENSEファイルが検出されず、ライセンスは未検証／未宣言となっています。

優先度区分: Helix-プライマリ